

Arduino App Lab

Guía de Arquitectura Modular

Cómo conectar HTML → JS → Python → Arduino



1. ¿Qué es Arduino App Lab?

Arduino App Lab es el entorno que permite crear aplicaciones que controlan un Arduino UNO R4 WiFi (u otras placas compatibles) desde una interfaz web. La idea central es separar las responsabilidades en capas bien definidas, de modo que puedas modificar una sin romper las otras.



Principio clave

Nunca mezcles capas. El HTML no habla directamente con el Arduino. El sketch no hace lógica de negocio. Cada archivo hace una sola cosa.

Las 4 capas del sistema

Archivo	Rol / Responsabilidad	Tecnología
index.html	Muestra la interfaz al usuario. Solo HTML puro, sin lógica.	<i>HTML</i>
script.js	Captura eventos del usuario y hace peticiones HTTP al servidor.	<i>JavaScript</i>
main.py	Servidor web + puente con el Arduino. Valida datos, llama al Bridge.	<i>Python</i>
sketch.ino	Corre en el Arduino. Recibe comandos del Bridge y controla hardware.	<i>C++ / Arduino</i>

2. El Flujo Completo de una Acción

Cuando el usuario hace clic en un botón, sucede lo siguiente en orden:

#	Capa	Qué sucede
1	index.html + script.js	El usuario hace clic. El onclick del botón llama a una función en script.js.
2	script.js → main.py	script.js hace un fetch() POST a una URL del servidor (ej: /enviar). Envía datos en formato JSON.
3	main.py	La función Python registrada en ese endpoint recibe el JSON, lo valida y llama a Bridge.call() con el comando.
4	sketch.ino	El Bridge llama al proveedor registrado en el Arduino. El sketch ejecuta la acción física (mover motor, encender LED, etc).
5	main.py → script.js	main.py devuelve un JSON de respuesta ({ok: true, ...}). script.js lo recibe en el .then() y actualiza la UI.

3. Cada Archivo en Detalle

3.1 index.html — La Interfaz

El HTML solo define qué ve el usuario. No contiene lógica. Cada elemento interactivo llama a una función de script.js mediante atributos como onclick.

index.html — Patrones de elementos interactivos

```
<!-- Botón que llama a la función enviarModo() en script.js -->
<button onclick="enviarModo(event, 'A')">□ Cara</button>

<!-- Para un control deslizante -->
<input type="range" min="0" max="7" oninput="setIntensidad(this.value)">

<!-- Para un input de texto -->
<input type="text" id="miTexto">
<button onclick="enviarTexto()">Enviar</button>
```



Regla de oro del HTML

El HTML nunca hace fetch() directamente. Solo llama funciones de script.js. Esto mantiene la lógica separada y reutilizable.

3.2 script.js — El Frontend

script.js es el intermediario entre la interfaz visual y el servidor Python. Su trabajo es: capturar eventos, preparar los datos en JSON, hacer el fetch(), y actualizar la UI con la respuesta.

script.js — Patrón fetch() estándar

```
// — Patrón estándar de una función de frontend —————

function enviarModo(e, modo) {
  // 1. Capturar el elemento que disparó el evento
  const boton = e.currentTarget;

  // 2. Hacer la petición HTTP al servidor Python
  fetch('/enviar', { // URL del endpoint en main.py
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ texto: modo }) // Datos en JSON
  })
  .then(res => res.json()) // Parsear la respuesta
  .then(data => {
    // 3. Actualizar la UI con la respuesta
    if (data.ok) mostrarEstado('✓ Enviado', false);
  })
  .catch(err => {
    // 4. Manejar errores de red
    mostrarEstado('✗ Error de conexión', true);
  });
}
```

Cómo extender: nuevos controles

script.js — Extensión para slider y lectura de datos

```
// Para un slider de intensidad:
function setIntensidad(valor) {
  fetch('/intensidad', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nivel: parseInt(valor) })
  }).then(res => res.json()).then(console.log);
}

// Para leer un valor del Arduino (GET):
function leerSensor() {
  fetch('/sensor') // GET, sin body
  .then(res => res.json())
  .then(data => {
    document.getElementById('resultado').textContent = data.valor;
  });
}
```

3.3 main.py — El Servidor Python

main.py es el núcleo del sistema. Sirve los archivos web, registra los endpoints de la API, valida los datos recibidos, y se comunica con el Arduino a través del Bridge.

main.py — Estructura completa

```
from arduino.app_bricks.web_ui import WebUI
from arduino.app_utils import App, Bridge, Logger

logger = Logger('mi-proyecto') # Nombre del proyecto para los logs
ui = WebUI(                     # Servidor web
    port=7000,
    assets_dir_path='./web',    # Carpeta con index.html, script.js, styles.css
)

# — Definir una función por cada endpoint —————

def enviar(payload: dict):      # payload = el JSON que mandó el fetch()
    texto = payload.get('texto', '') # Leer el campo 'texto'

    if not texto:               # Validar
        return {'ok': False, 'error': 'Falta el campo texto'}

    Bridge.call('recibir_comando', texto) # Llamar al Arduino

    return {'ok': True}         # Respuesta al frontend

# — Registrar endpoints —————
ui.expose_api('POST', '/enviar', enviar) # POST /enviar → función enviar()
ui.expose_api('GET', '/estado', leer_estado) # GET /estado → función leer_estado()

App.run() # Siempre al final
```

Los métodos del Bridge

Método	Cuándo usarlo	Ejemplo
Bridge.call(nombre, datos)	Enviar datos al Arduino y esperar que termine.	Bridge.call("motor", velocidad)
Bridge.notify(nombre, datos)	Enviar y no esperar (fuego y olvida). Útil para animaciones.	Bridge.notify("frame", datos)

3.4 sketch.ino — El Arduino

El sketch registra funciones como proveedores del Bridge y reacciona a los comandos de Python. El `loop()` generalmente queda vacío porque el Bridge maneja los eventos de forma asíncrona.

sketch.ino — Estructura completa

```
#include <Arduino_RouterBridge.h>
#include <Arduino_LED_Matrix.h> // Incluir la librería del hardware que uses

Arduino_LED_Matrix matrix;      // Instancia del hardware

// — Proveedores: funciones que Python puede llamar —————

void recibir_comando(String comando) { // Nombre = el que usas en Bridge.call()
    if (comando == "A") {
        // hacer algo con el hardware
    } else if (comando == "B") {
        // otra acción
    }
}

void apagar() { // Proveedor sin parámetros
    matrix.clear();
}

// — setup(): inicializar hardware y registrar proveedores ———

void setup() {
    matrix.begin();           // Iniciar el hardware
    matrix.clear();           // Estado inicial limpio

    Bridge.begin();           // SIEMPRE primero
    Bridge.provide("recibir_comando", recibir_comando); // Registrar
    Bridge.provide("apagar", apagar);
}

void loop() {
    // Vacío: el Bridge maneja los eventos de forma asíncrona
}
```

Tipos de datos en proveedores Bridge

Tipo C++	Cómo mandarlo desde Python	Ejemplo
<code>String</code> o <code>int</code>	Pasar el valor directamente	<code>Bridge.call("cmd", "hola")</code>
<code>std::vector<uint8_t></code>	Pasar una lista de Python	<code>Bridge.call("draw", [0,7,3,...])</code>

Tipo C++	Cómo mandarlo desde Python	Ejemplo
<code>std::array<uint32_t, N></code>	Lista de enteros de Python	<code>Bridge.call("anim", [h1,h2,h3,h4,dur])</code>
Sin parámetros (void)	Omitir el segundo argumento	<code>Bridge.call("apagar")</code>

4. Plantilla Modular — Copiar y Adaptar

Usa estos bloques como punto de partida para cualquier nuevo proyecto o funcionalidad. Solo necesitas cambiar los nombres y la lógica específica de tu hardware.

4.1 index.html — Plantilla

index.html — Plantilla base

```
<!DOCTYPE html>
<html>
<head>
  <title>Mi Proyecto Arduino</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Control de Hardware</h1>

  <!-- Botón simple -->
  <button onclick="enviarComando(event, 'ENCENDER') ">Encender</button>

  <!-- Slider (ej: velocidad de motor) -->
  <label>Velocidad: <span id="vel-val">0</span></label>
  <input type="range" min="0" max="255"
    oninput="setVelocidad(this.value) ">

  <!-- Display de datos del Arduino -->
  <p>Temperatura: <span id="temperatura">--</span> °C</p>
  <button onclick="leerSensor()">Actualizar sensor</button>

  <p id="estado"></p> <!-- Mensajes de feedback -->

  <script src="script.js"></script>
</body>
</html>
```

4.2 script.js — Plantilla

script.js — Plantilla base

```
// — Estado local de la UI —————
let botonActivo = null;

// — Función genérica de POST —————
function postJSON(endpoint, datos) {
  return fetch(endpoint, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(datos),
  }).then(res => {
```

```

        if (!res.ok) throw new Error('HTTP ' + res.status);
        return res.json();
    });
}

// — Enviar un comando con feedback visual —————
function enviarComando(e, comando) {
    const boton = e.currentTarget;
    if (botonActivo) botonActivo.classList.remove('activo');
    botonActivo = boton;
    boton.classList.add('activo');

    postJSON('/comando', { cmd: comando })
        .then(data => mostrarEstado('✔' + data.msg, false))
        .catch(err => { mostrarEstado('✗' + err.message, true);
    boton.classList.remove('activo'); });
}

// — Enviar un valor numérico (slider, input) —————
function setVelocidad(valor) {
    document.getElementById('vel-val').textContent = valor;
    postJSON('/velocidad', { nivel: parseInt(valor) })
        .catch(err => mostrarEstado('✗' + err.message, true));
}

// — Leer datos del Arduino (GET) —————
function leerSensor() {
    fetch('/sensor')
        .then(res => res.json())
        .then(data => { document.getElementById('temperatura').textContent =
data.valor; })
        .catch(err => mostrarEstado('✗' + err.message, true));
}

// — Mostrar mensajes de estado —————
function mostrarEstado(msg, esError) {
    const el = document.getElementById('estado');
    el.textContent = msg;
    el.style.color = esError ? '#ff4444' : '#00cc66';
    clearTimeout(el._t);
    el._t = setTimeout(() => el.textContent = '', 3000);
}

```

4.3 main.py — Plantilla

main.py — Plantilla base

```

from arduino.app_bricks.web_ui import WebUI
from arduino.app_utils import App, Bridge, Logger

logger = Logger('mi-proyecto')
ui = WebUI(port=7000, assets_dir_path='./web')

```



```

# — Endpoint: recibir un comando simple —————
def comando(payload: dict):
    cmd = payload.get('cmd', '')
    if not cmd:
        return {'ok': False, 'error': 'Falta cmd'}

    logger.info(f'Comando recibido: {cmd}')
    Bridge.call('ejecutar', cmd)      # nombre del proveedor en sketch.ino
    return {'ok': True, 'msg': f'Comando {cmd} enviado'}

# — Endpoint: enviar un valor numérico —————
def velocidad(payload: dict):
    nivel = payload.get('nivel', 0)
    nivel = max(0, min(255, int(nivel))) # Clamp 0-255
    Bridge.call('set_velocidad', nivel)
    return {'ok': True}

# — Endpoint: leer datos del Arduino —————
def sensor():
    # Si el Arduino devuelve datos, usar Bridge.call() con return value
    # Por ahora simulado:
    return {'ok': True, 'valor': 22.5}

# — Registrar todos los endpoints —————
ui.expose_api('POST', '/comando', comando)
ui.expose_api('POST', '/velocidad', velocidad)
ui.expose_api('GET', '/sensor', sensor)

App.run()

```

4.4 sketch.ino — Plantilla

sketch.ino — Plantilla base

```

#include <Arduino_RouterBridge.h>
// Incluir aquí las librerías del hardware que uses:
// #include <Servo.h>
// #include <Arduino_LED_Matrix.h>
// #include <Wire.h> // para I2C

// — Instancias de hardware —————
// Servo miServo;
// Arduino_LED_Matrix matrix;

// — Proveedores Bridge —————

void ejecutar(String cmd) {
    if (cmd == "ENCENDER") {
        // encender hardware
    } else if (cmd == "APAGAR") {
        // apagar hardware
    }
}

```

```
void set_velocidad(int nivel) {
    // miServo.write(nivel); // Ejemplo con servo
    // analogWrite(PIN_MOTOR, nivel); // Ejemplo con motor DC
}

// — setup —————
void setup() {
    // Inicializar hardware:
    // miServo.attach(9);
    // matrix.begin();

    Bridge.begin(); // SIEMPRE al final del setup
    Bridge.provide("ejecutar", ejecutar);
    Bridge.provide("set_velocidad", set_velocidad);
}

void loop() {
    // Vacío cuando usas Bridge
    // Si necesitas lógica continua (leer sensor cada N ms), ponla aquí
}
```

5. Ejemplos de Extensión

5.1 Agregar control de un Servo Motor

Un servo se controla con un ángulo (0–180°). Aquí está el cambio mínimo en cada capa:

index.html — Nuevo elemento

```
<!-- index.html: agregar slider de ángulo -->
<label>Ángulo: <span id="ang-val">90</span>°</label>
<input type="range" min="0" max="180" value="90"
  oninput="setAngulo(this.value)">
```

script.js — Nueva función

```
// script.js: nueva función
function setAngulo(valor) {
  document.getElementById('ang-val').textContent = valor;
  postJSON('/servo', { angulo: parseInt(valor) });
}
```

main.py — Nuevo endpoint

```
# main.py: nuevo endpoint
def servo(payload: dict):
    angulo = max(0, min(180, int(payload.get('angulo', 90))))
    Bridge.call('mover_servo', angulo)
    return {'ok': True}

ui.expose_api('POST', '/servo', servo)
```

sketch.ino — Nuevo proveedor

```
// sketch.ino: nuevo proveedor
#include <Servo.h>
Servo miServo;

void mover_servo(int angulo) {
  miServo.write(angulo);
}

void setup() {
  miServo.attach(9); // Pin 9
  Bridge.begin();
  Bridge.provide("mover_servo", mover_servo);
}
```

5.2 Leer un Sensor (datos del Arduino a la web)

Para leer datos en sentido inverso (Arduino → Web), la forma recomendada es que Python solicite el valor y lo devuelva al frontend:

main.py — Patrón de lectura de sensor

```
# main.py: endpoint que lee del Arduino
# Opción simple: el sketch guarda el último valor en una variable
# y Python la lee periódicamente. Aquí un ejemplo con almacenamiento local:

ultimo_valor = {'temp': 0.0} # Estado compartido

def actualizar_sensor(payload: dict):
    """El Arduino llama a esto via Bridge.notify para reportar valores."""
    ultimo_valor['temp'] = payload.get('temp', 0.0)
    return {'ok': True}

def leer_sensor():
    """El frontend llama a esto para obtener el último valor."""
    return {'ok': True, 'valor': ultimo_valor['temp']}

ui.expose_api('POST', '/actualizar_sensor', actualizar_sensor)
ui.expose_api('GET', '/sensor', leer_sensor)
```

5.3 Controlar un LED RGB (3 valores simultáneos)

index.html

```
// index.html
<input type="color" id="color-picker" oninput="setColor(this.value)">
```

script.js

```
// script.js
function setColor(hex) {
    // Convertir #RRGGBB a R, G, B separados
    const r = parseInt(hex.slice(1,3), 16);
    const g = parseInt(hex.slice(3,5), 16);
    const b = parseInt(hex.slice(5,7), 16);
    postJSON('/color', { r, g, b });
}
```

main.py

```
# main.py
def color(payload: dict):
    r = max(0, min(255, int(payload.get('r', 0))))
    g = max(0, min(255, int(payload.get('g', 0))))
    b = max(0, min(255, int(payload.get('b', 0))))
    Bridge.call('set_rgb', [r, g, b]) # Lista → vector<uint8_t> en C++
    return {'ok': True}
```

```
ui.expose_api('POST', '/color', color)
```

sketch.ino — LED RGB con vector

```
// sketch.ino
#define PIN_R 9
#define PIN_G 10
#define PIN_B 11

void set_rgb(std::vector<uint8_t> valores) {
    if (valores.size() < 3) return;
    analogWrite(PIN_R, valores[0]);
    analogWrite(PIN_G, valores[1]);
    analogWrite(PIN_B, valores[2]);
}

void setup() {
    pinMode(PIN_R, OUTPUT);
    pinMode(PIN_G, OUTPUT);
    pinMode(PIN_B, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_rgb", set_rgb);
}
```

6. Errores Comunes y Cómo Resolverlos

Síntoma	Causa probable	Solución
fetch() da error de red / CORS	El servidor Python no está corriendo o usa otro puerto.	Verificar que App.run() esté al final de main.py y que el puerto sea 7000.
El Arduino no reacciona al comando	El nombre en Bridge.call() no coincide con el de Bridge.provide().	Verificar que el string sea idéntico en main.py y sketch.ino (incluyendo mayúsculas).
El sketch compila pero no hace nada	Bridge.begin() fue llamado antes de inicializar el hardware.	Inicializar todo el hardware primero, Bridge.begin() al final del setup().
Python devuelve error 500	Excepción no manejada en la función del endpoint.	Envolver el código en try/except y devolver {'ok': False, 'error': str(e)}.
Los archivos HTML/CSS/JS no cargan	assets_dir_path apunta a la carpeta equivocada.	Verificar que assets_dir_path='./web' y que index.html esté dentro de esa carpeta.

7. Checklist para Agregar una Nueva Funcionalidad

☐ Proceso en 4 pasos

Cada vez que quieras agregar control de algo nuevo (un sensor, motor, LED, etc.), sigue este orden.

✓	Paso	Qué hacer
1	sketch.ino	Escribe la función void en C++. Llama al hardware dentro. Regístrala con Bridge.provide("nombre", funcion) en setup().
2	main.py	Escribe la función Python que recibe el payload, valida, y llama a Bridge.call("nombre", datos). Regístrala con ui.expose_api().
3	script.js	Escribe la función JS que hace fetch('/tu-endpoint', {...}). Actualiza la UI con la respuesta en el .then().

✓	Paso	Qué hacer
4	index.html	Agrega el elemento HTML (botón, slider, input) con el onclick o oninput que llame a tu función de script.js.